

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

КУРСОВАЯ РАБОТА
по дисциплине «Программирование»
Тема: Обработка изображения

Студентка гр. 4342

Киселева А.Д.

Преподаватель

Виноградова Е.В.

Санкт-Петербург

2025

ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ

Студентка Киселева А.Д.

Группа 4342

Тема работы: Обработка изображения

Исходные данные:

Требуется создать CLI-утилиту для обработки PNG изображения на языке C с использованием библиотеки `libpng` и применением `getopt`, которая поддерживает рисование квадрата, перемещение частей изображения в заданной прямоугольной области, смену наиболее часто встречаемого цвета на выбранный пользователем цвет.

Содержание пояснительной записки:

«Аннотация», «Содержание», «Введение», «Цель», «Задачи», «Описание программы», «Примеры работы программы», «Заключение», «Список использованных источников»

Предполагаемый объем пояснительной записки:

Не менее 10 страниц.

Дата выдачи задания: 07.03.2025

Дата сдачи реферата: 23.05.2025

Дата защиты реферата: 25.05.2025

Студентка

Киселева А.Д.

Преподаватель

Виноградова Е.В.

АННОТАЦИЯ

Курсовая работа посвящена разработке консольной утилиты для обработки PNG-изображений с использованием библиотеки libpng. Программа поддерживает три основные функции: рисование квадрата с заданными параметрами, перестановку четырех частей выбранной области изображения и замену наиболее часто встречающегося цвета на указанный. Утилита реализована в соответствии с требованиями CLI, включает обработку ошибок, справку и поддержку как полных, так и сокращенных аргументов командной строки.

SUMMARY

The course work is devoted to the development of a console utility for processing PNG-images using the libpng library. The program supports three main functions: drawing a square with preset parameters, rearranging four parts of the selected image area, and replacing the most common color with the specified one. The utility is implemented in accordance with the requirements of the CLI, includes error handling, help, and support for both full and abbreviated command line arguments.

СОДЕРЖАНИЕ

Введение	5
1. Описание программы	6
1.1. Организация хранения данных	6
1.2. Общая логика работы программы	6
1.3. Реализация интерфейса командной строки	7
1.4. Считывание и запись изображения	8
1.5. Функционал рисования квадрата	9
1.6. Функционал обмена частей изображения	10
1.7. Функционал замены частотного цвета	11
1.8. Обработка ошибок	12
1.9. Вывод справочной информации	12
2. Примеры работы программы	13
2.1. Обработка корректных запросов	13
2.2. Обработка ошибочных случаев	14
Заключение	17
Список использованных источников	18
Приложение А. Тестирование	19
Приложение Б. Код программы	23

ВВЕДЕНИЕ

Цель: разработать консольную программу для обработки PNG-изображений, обеспечивающую выполнение заданных графических операций с корректной обработкой входных данных и ошибок.

Задачи:

1. Организовать обработку аргументов командной строки
2. Первично обработать входное изображение: реализовать функции чтения и записи, проверить корректность получаемого файла
3. Реализовать функции операций с изображением
4. Настроить обработку ошибок
5. Протестировать и отладить программу

1. ОПИСАНИЕ ПРОГРАММЫ

1.1. Организация хранения данных

Для хранения данных о вводимом изображении используется структура Png, которая содержит поля типа int, предназначенные для длины, ширины и количества проходов для полной загрузки изображения, поля типа png_byte для цветотипа и глубины цвета, поле типа png_bytep* для хранения массива пикселей, поля типа png_structp и png_infor, содержащие информацию об изображении.

Для удобства созданы структуры RGB, хранящая информацию о каждой компоненте цвета пикселя, и point, предназначенная для записи координат точки.

Для хранения данных об ожидаемых флагах каждой функции реализованы структуры square_data, exchange_data и change_color.

Первая состоит из поля типа struct point* для задания координат левой верхней точки, полей типа int для размера стороны и толщины линий, поля типа bool для получения информации о наличии заливки, полей типа struct RGB* для цвета линий и заливки. Вторая состоит из полей типа struct point*, задающих верхнюю левую и правую нижнюю точки, поля типа char*, отвечающего за тип перемещения частей изображения. Третья состоит из поля типа struct RGB*, предназначенного для хранения цвета, в который будут перекрашены все пиксели наиболее часто встречаемого цвета.

Для отслеживания получения данных о вызываемых операциях используется структура options_flags, состоящая из полей типа int и поля типа enum error_types, которое отвечает за наличие ошибок.

1.2. Общая логика работы программы

Программа разбита на функции, выполняющие отдельные задачи.

В основной функции `main` выделяется память для используемых структур, происходит считывание аргументов командной строки и заполнение полей соответствующих структур, вызываются функции чтения, обработки и записи изображения. В конце освобождается динамически выделенная память.

Более подробное описание каждой функции изложено в следующих подразделах.

1.3. Реализация интерфейса командной строки

Чтение аргументов происходит при помощи `getopt_long`, поскольку данная функция позволяет обрабатывать как длинные, так и короткие флаги. При получении неизвестного флага функция возвращает ошибку аргумента. При успешном считывании опции проверяется корректность полученного аргумента, обновляются поля отслеживающей структуры и структуры соответствующей функции, которая может содержать данный аргумент. Для опций, не принимающих на вход аргументы, выполняется проверка корректности следующего аргумента командной строки (является ли он флагом). Проводится дополнительная проверка корректности имени входного изображения, когда оно подается последним аргументом без соответствующего флага.

Ожидаемые на вход флаги:

- 1 `--input, -i`: задает имя входного изображения. Если флаг отсутствует, то ожидается, что имя изображения подано последним аргументом.
- 2 `--output, -o`: задает имя выходного изображения. Если флаг отсутствует, имя выходного изображения по умолчанию задается как «out.png».
- 3 `--info`: выводится информация об изображении.

- 4 `--help, -h`: выводится справка, содержащая информацию о флагах и их аргументах.
- 5 `--square, -S`: рисование квадрата. Определяется следующими опциями:
- 5.1 `--left_up, -l`: координатами левого верхнего угла. Значение задаётся в формате «left.up», где left – координата по x, up – координата по y.
 - 5.2 `--side_size, -s`: размером стороны. На вход принимает число больше 0.
 - 5.3 `--thickness, -t`: толщиной линий. На вход принимает число больше 0.
 - 5.4 `--color, -C`: цветом линий. Цвет задаётся строкой «rrr.ggg.bbb», где rrr/ggg/bbb – числа в диапазоне [0; 255], задающие цветовую компоненту.
 - 5.5 `--fill, -f`: наличием заливки. Если флаг подан, квадрат будет залит, иначе — остается пустым.
 - 5.6 `--fill_color, -c`: цветом заливки. Работает аналогично флагу — color.
- 6 `--exchange, -e`: перемещение частей изображения. Определяется следующими опциями:
- 6.1 `--left_up, -l`: координатами левого верхнего угла области. Работает аналогично соответствующему флагу, описанному в п. 5.1.
 - 6.2 `--right_down, -r`: координатами правого нижнего угла области. Работает аналогично флагу `--left_up`.
 - 6.3 `--exchange_type, -T`: способом обмена частей. Возможные значения: «clockwise» - по часовой стрелке, «counterclockwise» - против часовой стрелки, «diagonals» - по диагоналям.

7 --freq_color, -F: поиск наиболее часто встречаемого цвета и замена на другой заданный цвет. Определяется следующими опциями:

7.1 --color, -C: цвет в который надо перекрасить самый часто встречаемый цвет. Работает аналогично соответствующему флагу, описанному в п. 5.4.

После считывания аргументов для проверки корректности введенных данных вызывается функция `check_input_data`, в которую подается структура, отслеживающая наличие тех или иных флагов. Проверяется наличие всех необходимых аргументов для требуемой функции и отсутствие лишних.

1.4. Считывание и запись изображения

Работа с изображением проводится с помощью функций библиотеки `libpng`.

Функция считывания изображения принимает на вход указатель на структуру `Png` для хранения изображения, путь к читаемому изображению, переменную для записи возможных ошибок. Файл с изображением открывается в бинарном режиме чтения, происходит считывание заголовка `header` и проверка его соответствия сигнатуре `png`. Проводится инициализация структур для чтения и хранения информации об изображении, также осуществляется проверка успешности их создания. Происходит подготовка к чтению файла: настраивается ввод, пропускается проверка сигнатуры, так как она уже была сделана выше, считываются метаданные изображения для дальнейшей работы. Далее извлекаются основные параметры (ширина, высота, цветотип, глубина цвета, количество проходов для полной загрузки) и создается массив указателей на строки изображения, в каждую из них записывается информация о пикселях, из которых она состоит. Функция возвращает значение типа `int` равное 1, если чтение изображения произошло успешно, иначе — 0.

Функция записи изображения принимает на вход указатель на структуру Png для хранения изображения, путь к файлу, в который будет записано изображение, переменную для записи возможных ошибок. Файл открывается в режиме бинарной записи, инициализируются структуры для записи и хранения информации об изображении, осуществляется проверка успешности их создания. Устанавливаются основные параметры изображения. Заголовок и данные изображения записываются в переданный файл, открытый в бинарном режиме записи. Очищаются структуры изображения.

При наличии каких-либо ошибок в консоль печатается сообщение, код ошибки записывается в переданную переменную.

1.5. Функционал рисования квадрата

Для рисования квадрата создана void-функция, принимающая на вход указатель на структуру с введенными пользователем данными и указатель на структуру, содержащую информацию об изображении. Рисуются две горизонтальные и две вертикальные линии. Предполагается, что ширина линий отсчитывается равномерно в обе стороны от центра. Чтобы корректно нарисовать углы квадрата, обе координаты сдвинуты на половину толщины. Далее, если требуется, выполняется заливка полученного квадрата.

Для рисования вертикальных и горизонтальных линий создана отдельная void-функция, принимающая на вход координаты начальной точки, длину, толщину и цвет линий, а также указатель на структуру, содержащую информацию об изображении. Пиксели, лежащие на линии заданной толщины и длины, проходят проверку на принадлежность изображению и перекрашиваются в заданный цвет, если она успешна.

Для заливки квадрата создана отдельная void-функция, принимающая на вход координаты начальной точки заливки (левого верхнего угла внутреннего

квадрата), длины сторон по оси x и по оси y (для возможного использования функции для заливки прямоугольника), цвет и указатель на структуру, хранящую информацию об изображении. Пиксели, принадлежащие внутреннему квадрату и находящиеся в пределах изображения, перекрашиваются в заданный цвет.

1.6. Функционал обмена частей изображения

Согласно заданию, выбранная пользователем прямоугольная область делится на 4 части, после чего фрагменты перемещаются в соответствие с заданным способом обмена.

Создана `void`-функция, принимающая на вход указатель на структуру, содержащую введенные данные, указатель на структуру, хранящую информацию об изображении, переменную для записи возможных ошибок. Производится проверка корректности ввода координат прямоугольной области. При некорректном положении точек их координаты меняются местами (левая верхняя становится правой нижней, правая нижняя — левой верхней). Далее сохраняются данные о каждой из четырех частей выбранной области. В зависимости от способа обмена выполняется перестановка фрагментов области. После всех операций освобождается память, выделенная для хранения частей области.

Для получения данных о фрагментах создана отдельная `void`-функция, принимающая на вход размеры сторон фрагмента по оси x и по оси y , координаты на изображении, откуда будет произведено копирование, указатель на массив, в который будет сохранен фрагмент, указатель на структуру, хранящую данные об изображении, переменную для записи возможных ошибок. Внутри функции выделяется память для пикселей части изображения, проводится проверка принадлежности текущего пикселя исходному

изображению. В случае успеха пиксель изображения сохраняется в массив фрагмента.

Для перестановки частей области создана отдельная void-функция вставки, которая принимает на вход координаты верхнего левого угла, с которого начинается вставка, длину и высоту вставляемого фрагмента, массив, в котором хранятся пиксели вставляемой части, указатель на структуру, содержащую информацию об исходном изображении. Внутри функции, после проверки на принадлежность текущего пикселя исходному изображению, выполняется перезапись этого пикселя в соответствие с данными из вставляемой части.

1.7. Функционал замены частого цвета

Для замены наиболее часто встречаемого цвета реализована void-функция, принимающая на вход указатель на структуру с введенными данными, указатель на структуру, содержащую информацию об изображении, переменную для записи возможных ошибок. Внутри функции создается массив для хранения количества вхождений каждого цвета, а также переменная для подсчёта максимального количества вхождений. При проходе по изображению вычисляется уникальный индекс цвета, опираясь на то, что цветовые составляющие пикселя хранятся последовательно и занимают 3 байта памяти (в реализации для png), и увеличивается значение счетчика текущего цвета на 1. Затем происходит сравнение этого счетчика со счетчиком максимального количества вхождений. Если текущий счетчик больше, то обновляется главный счетчик и сохраняются компоненты цвета рассматриваемого пикселя. Далее происходит повторный проход по изображению, в котором осуществляется поиск пикселей, чьи цветовые составляющие совпадают с сохраненными ранее, и замена их цвета на заданный пользователем.

1.8. Обработка ошибок

Для удобства задания ошибок создано перечисление `error_types`, в котором каждому типу ошибки соответствует свой код из диапазона [40; 49].

Для случая корректного выполнения программы создан флаг равный 0.

В структуре `options_flags`, предназначенной для отслеживания введенных пользователем опций, создано поле типа `error_types` для отслеживания ошибок, изначально равное 0. Это поле подается в функции, в которых могут быть встречены критические случаи. В случае отлова ошибки главная часть кода функции не выполняется, полю присваивается соответствующее значение и выводится сообщение в консоль.

1.9 Вывод справочной информации

Для вывода справочной информации о поданном изображении создана `void`-функция, принимающая на вход указатель на структуру, хранящую данные об изображении.

Для вывода справочной информации о доступных опциях создана `void`-функция, не принимающая аргументов.

2. ПРИМЕРЫ РАБОТЫ ПРОГРАММЫ

2.1. Вывод справочной информации

1. Проверка опции вывода справки

Таблица 1

Ввод	<code>./a.out --help</code>
Вывод	см. Приложение А. Таблица 13

Таблица 2

Ввод	<code>./a.out -h input.png</code>
Вывод	см. Приложение А. Таблица 13

2. Проверка опции рисования квадрата

Таблица 3 — см. Приложение А. Рисунок 2.1.1., Рисунок 2.1.2.

Ввод	<code>./a.out --square --left_up 50.75 --side_size 234 --thickness 15 --color 0.255.120 --output cool.png doll.png</code>
Вывод	Course work for option 4.16, created by Alina Kiseleva

Таблица 4 — см. Приложение А. Рисунок 2.2.1, Рисунок 2.2.2.

Ввод	<code>./a.out --square --left_up 200.275 --side_size 400 --thickness 7 --color 206.91.120 --fill --fill_color 0.255.50 --output cool.png input.png</code>
------	---

Вывод	Course work for option 4.16, created by Alina Kiseleva
-------	--

3. Проверка опции перемещения частей области

Таблица 5 — см. Приложение А. Рисунок 3.1.1, Рисунок 3.1.2.

Ввод	<code>./a.out --exchange --left_up 150.259 --right_down 456.150 --exchange_type diagonals --input radio.vQh</code>
Вывод	Course work for option 4.16, created by Alina Kiseleva

Таблица 6 — см. Приложение А. Рисунок 3.2.1, Рисунок 3.2.2.

Ввод	<code>./a.out --exchange --left_up 500.375 --right_down 0.0 --exchange_type clockwise --input milk.XyA</code>
Вывод	Course work for option 4.16, created by Alina Kiseleva

4. Проверка опции замены частого цвета

Таблица 7 — см. Приложение А. Рисунок 4.1.1, Рисунок 4.1.2.

Ввод	<code>./a.out --freq_color --color 227.37.107 car.nST</code>
Вывод	Course work for option 4.16, created by Alina Kiseleva

2.2. Обработка ошибочных случаев

Таблица 8

Ввод	<code>./a.out --balerina cappuchino</code>
Вывод	<code>./a.out: unrecognized option '--balerina'</code> Введенного флага не существует. Входной файл отсутствует в текущей директории

Таблица 9

Ввод	<code>./a.out --freq_color --color 1000minus7 car.nST</code>
Вывод	Цвет задан неверно

Таблица 10

Ввод	<code>./a.out --freq_color --color 227.37.107 out.png</code>
Вывод	Имена входного и выходного файлов совпадают

Таблица 11

Ввод	<code>./a.out --info weather car.nST</code>
Вывод	Опция <code>--info</code> не принимает аргументов. <code>weather</code> и последующие аргументы будут проигнорированы.

Таблица 12

Ввод	<code>./a.out -o -i car.nST</code>
Вывод	Не введено имя выходного файла

ЗАКЛЮЧЕНИЕ

В рамках курсовой работы создана CLI-утилита для обработки PNG изображения, поддерживающая функции рисования квадрата, перестановки фрагментов изображения заданным образом, замены наиболее часто встречаемого цвета. Реализована обработка аргументов командной строки, устранены возможные ошибки. Утилита готова к практическому применению, её архитектура позволяет легко расширять функционал.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Базовые сведения к выполнению курсовой работы по дисциплине «Программирование». Второй семестр / сост.: М. М. Заславский, А. А. Лисс, А. В. Гаврилов и др.. СПб.: Изд-во СПбГЭТУ «ЛЭТИ», 2024. 36 с.
2. A description on how to use and modify libpng. URL: <http://www.libpng.org/pub/png/libpng-1.2.5-manual.html#section-2>
3. Керниган Б.В., Ричи Д.М. Язык С. Спб.: "Невский Диалект", 2001. 352 с.

ПРИЛОЖЕНИЕ А

ТЕСТИРОВАНИЕ

Таблица 13

Course work for option 4.16, created by Alina Kiseleva

Общие опции:

- input -i имя входного изображения
- output -o переопределить имя выходного изображения (по умолчанию out.png)
- info информация об изображении
- help -h справка

Флаги функций:

- square -S рисование квадрата
- left_up -l координаты левого верхнего угла
- side_size -s размер стороны
- thickness -t толщина линий
- color -C цвет линий / цвет для изменения текущего
- fill -f заливка
- fill_color -c цвет заливки
- exchange -e меняет местами 4 куска заданной области
- left_up -l координаты левого верхнего угла области
- right_down -r координаты правого нижнего угла области
- exchange_type -T способ обмена частей
- freq_color -F поиск наиболее часто встречаемого цвета и замена его на другой



Рисунок 2.1.1



Рисунок 2.1.2

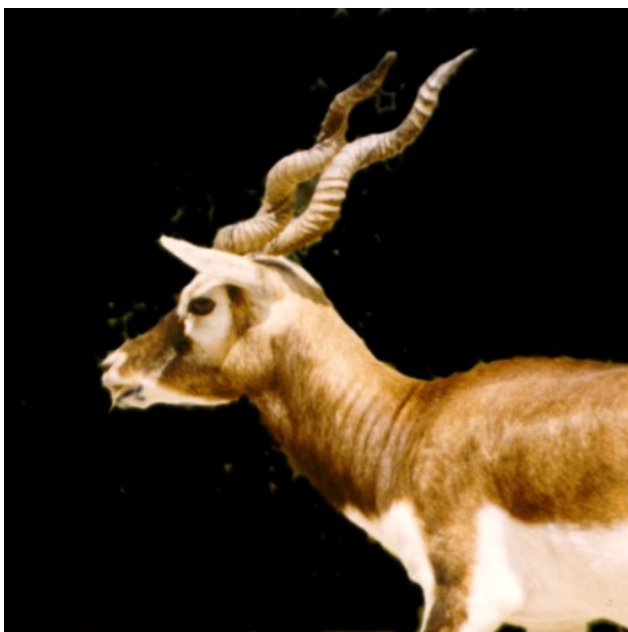


Рисунок 2.2.1

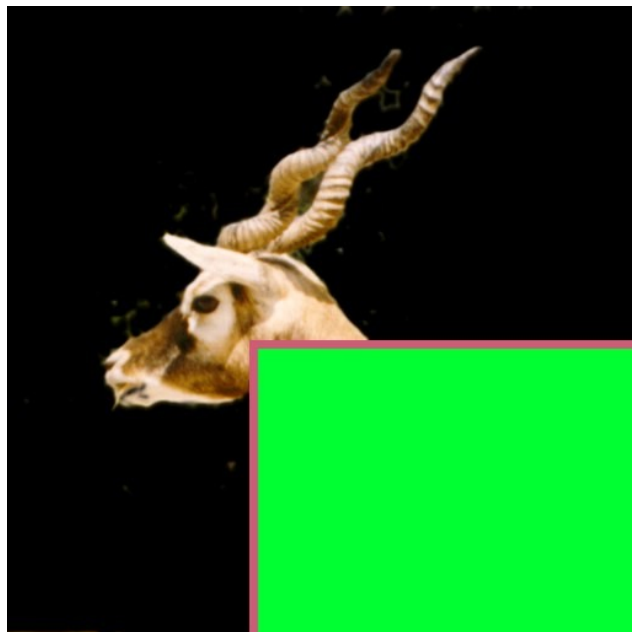


Рисунок 2.2.2



Рисунок 3.1.1



Рисунок 3.1.2



Рисунок 3.2.1



Рисунок 3.2.2



Рисунок 4.1.1



Рисунок 4.1.2

ПРИЛОЖЕНИЕ Б

КОД ПРОГРАММЫ

```
#include <stdio.h>

#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
#include <getopt.h>
#include <unistd.h>
#include <png.h>
#define SZ_OPTSTR 30
#define OPT_INFO 0
#define NUM_COORDS 2
#define SZ_OUTPUT 8
#define SZ_INPUT 8
#define NUM_FLAGS 17
#define LEN_HEADER 8
#define NUM_RGB 3
#define NUM_COLORS_RGB 0x1000000 // 256^3
#define IMG_READ 1
#define IMG_NOT_READ 0
#define CORRECT_COLOR 1
#define WRONG_COLOR 0
#define FUNC_11 0xFF00 // 0b1111111100000000
#define FUNC_12 0xFC00 // 0b1111110000000000
#define FUNC_2 0xA0E0 // 0b1010000011100000
#define FUNC_3 0x8410 // 0b1000010000010000
#define FUNC_4 0x800E // 0b1000000000001110

enum error_types{
    ERR_OK = 0,
    ERR_MEM = 40,
    ERR_OPT,
    ERR_COLOR,
    ERR_SIGNATURE,
    ERR_COORDS,
    ERR_IMG_TYPE,
    ERR_INPUT_NAME,
    ERR_OUTPUT_NAME,
```

```

        ERR_EXCH_TYPE,
        ERR_IMG_READ
};

struct Png{
    int width, height;
    png_byte color_type;
    png_byte bit_depth;
    png_structp png_ptr;
    png_infop info_ptr;
    int number_of_passes;
    png_bytep* row_pointers;
};

struct RGB{
    int R;
    int G;
    int B;
};

struct point{
    int x;
    int y;
};

struct contrast_data{
    double alpha;
    int beta;
};

struct square_data{
    struct point* left_up;
    int side_size;
    int thickness;
    struct RGB* color;
    bool fill;
    struct RGB* fill_color;
};

struct exchange_data{
    struct point* left_up;

```



```

        struct point* right_down;
        char* type;
};

struct change_color{
    struct RGB* new_color;
};

struct options_flags{
    unsigned iname_flag:1;

    unsigned sqr_flag:1;
    unsigned lft_up_flag:1;
    unsigned sd_sz_flag:1;
    unsigned thkns_flag:1;
    unsigned clr_flag:1;
    unsigned fill_flag:1;
    unsigned fill_clr_flag:1;
    unsigned exchg_flag:1;
    unsigned rt_dwn_flag:1;
    unsigned exchg_t_flag:1;
    unsigned fq_clr_flag:1;

    unsigned contr_flag:1;
    unsigned a_flag:1;
    unsigned b_flag:1;

    unsigned err:1;
};

union get_flags{
    struct options_flags inp_flags;
    struct{
        unsigned iname_bit:1;
        unsigned square_bit:1;
        unsigned lft_up_bit:1;
        unsigned sd_sz_bit:1;
        unsigned thkns_bit:1;
        unsigned clr_bit:1;
        unsigned fill_bit:1;
        unsigned fill_clr_bit:1;
    };
};

```

```

        unsigned exchg_bit:1;
        unsigned rt_dwn_bit:1;
        unsigned exchg_t_bit:1;
        unsigned fq_clr_bit:1;
        unsigned contr_bit:1;
        unsigned a_bit:1;
        unsigned b_bit:1;
        unsigned err_bit:1;
    }bits;
};

int check_input_data(struct options_flags flags){
    int checker = 0;

    int expected_func_11 = FUNC_11;
    int expected_func_12 = FUNC_12;
    int expected_func_2 = FUNC_2;
    int expected_func_3 = FUNC_3;
    int expected_func_4 = FUNC_4;

    union get_flags my_flags;
    my_flags.inp_flags = flags;

    unsigned short* bits = (unsigned
short*)&my_flags.bits;
    int received_flags = 0;

    for (int i=0; i<16; i++)
        received_flags |= ((*bits >> i) & 1) <<
(15 - i);

    if (((received_flags & expected_func_11) ==
expected_func_11) || ((received_flags & expected_func_12)
== expected_func_12))
        checker = 1;
    else if ((received_flags & expected_func_2) ==
expected_func_2)
        checker = 2;
    else if ((received_flags & expected_func_3) ==
expected_func_3)
        checker = 3;

```

```

        else if ((received_flags & expected_func_4) ==
expected_func_4)
            checker = 4;

        return checker;
    }

int check_color(int R, int G, int B){
    return ((R>=0 && R<256) && (G>=0 && G<256) &&
(B>=0 && B<256)) ? CORRECT_COLOR : WRONG_COLOR;
}

void print_help(){
    printf("\n\tОбщие опции:\n");
    printf("--input -i имя входного изображения\n");
    printf("--output -o переопределить имя выходного
изображения (по умолчанию out.png)\n");
    printf("--info информация об изображении\n");
    printf("--help -h справка\n\n");
    printf("\tФлаги функций:\n");
    printf("--square -S рисование квадрата\n");
    printf("--left_up -l координаты левого верхнего
угла\n");
    printf("--side_size -s размер стороны\n");
    printf("--thickness -t толщина линий\n");
    printf("--color -C цвет линий / цвет для
изменения текущего\n");
    printf("--fill -f заливка\n");
    printf("--fill_color -c цвет заливки\n");
    printf("--exchange -e меняет местами 4 куска
заданной области\n");
    printf("--left_up -l координаты левого верхнего
угла области\n");
    printf("--right_down -r координаты правого
нижнего угла области\n");
    printf("--exchange_type -T способ обмена частей\
n");
    printf("--freq_color -F поиск наиболее часто
встречаемого цвета и замена его на другой\n\n");
}

```

```

void print_png_info(struct Png* img){
    printf("Информация об изображении: \n\n");
    printf("Ширина: %d пикселей\n", img->width);
    printf("Высота: %d пикселей\n", img->height);
    printf("Цветовая модель: ");

    switch(img->color_type){
        case PNG_COLOR_TYPE_GRAY:
            printf("grayscale (1 канал)\n");
            break;
        case PNG_COLOR_TYPE_RGB:
            printf("RGB (3 канала)\n");
            break;
        case PNG_COLOR_TYPE_GRAY_ALPHA:
            printf("grayscale + alpha (2
канала)\n");
            break;
        case PNG_COLOR_TYPE_RGBA:
            printf("RGBA (4 канала)\n");
            break;
        default:
            printf("неизвестный тип\n");
            break;
    }

    printf("Глубина цвета: %d бит/пиксель\n", img->bit_depth);
    printf("Количество проходов для полной загрузки: %d\n", img->number_of_passes);
}

int read_image(struct Png* image, char* filename, enum error_types* err){
    FILE* file = fopen(filename, "rb");
    if (file){
        png_byte header[LEN_HEADER] = {0};
        fread(header, 1, 8, file);

        if (!png_sig_cmp(header, 0, 8)){

```

```

        image->png_ptr =
png_create_read_struct(PNG_LIBPNG_VER_STRING, NULL, NULL,
NULL);

        image->info_ptr =
png_create_info_struct(image->png_ptr);

        if (image->png_ptr && image-
>info_ptr){
            png_init_io(image-
>png_ptr, file);
            png_set_sig_bytes(image-
>png_ptr, LEN_HEADER);
            png_read_info(image-
>png_ptr, image->info_ptr);
            image->width =
png_get_image_width(image->png_ptr, image->info_ptr);
            image->height =
png_get_image_height(image->png_ptr, image->info_ptr);
            image->color_type =
png_get_color_type(image->png_ptr, image->info_ptr);
            image->bit_depth =
png_get_bit_depth(image->png_ptr, image->info_ptr);
            image->number_of_passes =
png_set_interlace_handling(image->png_ptr);
            png_read_update_info(image->png_ptr, image->info_ptr);

            image->row_pointers =
(png_bytep*)malloc(sizeof(png_bytep)*image->height);
            if (image->row_pointers){
                for (int y = 0; y
< image->height; y++){
                    image-
>row_pointers[y] =
(png_byte*)malloc(png_get_rowbytes(image->png_ptr, image-
>info_ptr));
                    if
(image->row_pointers[y] == NULL)
*err = ERR_MEM;
                }

```

```

ERR_OK)
    if (*err ==
png_read_image(image->png_ptr, image->row_pointers);
    else *err =
ERR_IMG_READ;

    } else {
        printf("Ошибка
памяти\n");
        *err = ERR_MEM;
    }

    } else {
        printf("Ошибка выделения
памяти для структуры на чтение PNG\n");
        *err = ERR_MEM;
    }

    png_destroy_read_struct(&(image-
>png_ptr), &(image->info_ptr), NULL);
    png_destroy_info_struct(image-
>png_ptr, &(image->info_ptr));

    } else {
        printf("Входное изображение не
соответствует PNG формату\n");
        *err = ERR_SIGNATURE;
    }

    fclose(file);

    } else {
        printf("Входной файл отсутствует в
текущей директории\n");
        *err = ERR_MEM;
    }

    return (*err == ERR_OK) ? IMG_READ :
IMG_NOT_READ;
}

```

```

void write_image(struct Png* image, char* filename, enum
error_types* err){
    FILE* file = fopen(filename, "wb");
    if (file){
        image->png_ptr =
png_create_write_struct(PNG_LIBPNG_VER_STRING, NULL,
NULL, NULL);
        image->info_ptr =
png_create_info_struct(image->png_ptr);

        if (image->png_ptr && image->info_ptr){
            png_init_io(image->png_ptr,
file);

            png_set_IHDR(image->png_ptr,
                        image->info_ptr,
                        image->width,
                        image->height,
                        image->bit_depth,
                        image->color_type,
                        PNG_INTERLACE_NONE,
                        PNG_COMPRESSION_TYPE_BASE,
                        PNG_FILTER_TYPE_BASE);
            png_write_info(image->png_ptr,
image->info_ptr);
            png_write_image(image->png_ptr,
image->row_pointers);
            png_write_end(image->png_ptr,
NULL);

            for (int y = 0; y < image->
height; y++)
                free(image->
row_pointers[y]);
            free(image->row_pointers);
            fclose(file);
        } else {

```

```

        printf("Ошибка выделения памяти
для структуры на запись PNG\n");
        *err = ERR_MEM;
    }

    png_destroy_info_struct(image->png_ptr,
&(image->info_ptr));
    png_destroy_write_struct(&(image-
>png_ptr), &(image->info_ptr));

    } else {
        printf("Ошибка выходного файла\n");
        *err = ERR_MEM;
    }
}

void draw_line(int x, int y, int side, int thick, struct
RGB* color, struct Png* image){
    for (int i=y; i<y+thick; i++){
        png_byte* row = image->row_pointers[i];
        for (int j=x; j<x+side; j++){
            png_byte* pixel = &(row[j*3]);

            if ((j >= 0 && j < image->width)
&& (i >= 0 && i < image->height)){
                pixel[0] = color->R;
                pixel[1] = color->G;
                pixel[2] = color->B;
            }
        }
    }
}

void fill_square(int x, int y, int side_x, int side_y,
struct RGB* fill_color, struct Png* image){
    for (int i=y; i<y+side_y; i++){
        png_byte* row = image->row_pointers[i];
        for (int j=x; j<x+side_x; j++){
            png_byte* pixel = &(row[j*3]);

```



```

                                if ((j >= 0 && j < image->width)
&& (i >= 0 && i < image->height)){
                                    pixel[0] = fill_color->R;
                                    pixel[1] = fill_color->G;
                                    pixel[2] = fill_color->B;
                                }
                            }
                    }
}

```

```

void draw_square(struct square_data* my_square, struct
Png* image){
    int x = my_square->left_up->x;
    int y = my_square->left_up->y;
    int thick = my_square->thickness;
    int side = my_square->side_size;

    draw_line(x-(thick/2), y-(thick/2), side+thick,
thick, my_square->color, image);
    draw_line(x-(thick/2), y-(thick/2)+side,
side+thick, thick, my_square->color, image);
    draw_line(x-(thick/2), y-(thick/2), thick,
side+thick, my_square->color, image);
    draw_line(x-(thick/2)+side, y-(thick/2), thick,
side+thick, my_square->color, image);

    if (my_square->fill)
        fill_square(x+(thick/2)+(thick%2), y+
(thick/2)+(thick%2), side-thick, side-thick, my_square-
>fill_color, image);
}

```

```

void get_piece(int len_x, int len_y, int img_x, int
img_y, png_bytep* piece, struct Png* image, enum
error_types* err){
    for (int y=0; y<len_y; y++){
        piece[y] = (png_byte*)calloc(len_x*3,
sizeof(png_byte));

        if (piece[y]){
            for (int x=0; x<len_x; x++){

```

```

        png_byte* dst_pix =
&(piece[y][x*3]);
        png_byte* src_pix =
&(image->row_pointers[img_y+y][(img_x+x)*3]);

        if ((img_x+x >= 0 &&
img_x+x < image->width) && (img_y+y >= 0 && img_y+y <
image->height)){
            dst_pix[0] =
src_pix[0];
            dst_pix[1] =
src_pix[1];
            dst_pix[2] =
src_pix[2];
        }
    }
} else {
    printf("Ошибка памяти\n");
    *err = ERR_MEM;
}
}

void paste_piece(int x, int y, int len_x, int len_y,
png_bytep* piece, struct Png* image){
    for (int i=0; i<len_y; i++){
        for (int j=0; j<len_x; j++){
            png_byte* dst_pix = &(image-
>row_pointers[y+i][(x+j)*3]);
            png_byte* src_pix = &(piece[i]
[j*3]);

            if ((x+j >= 0 && x+j < image-
>width) && (y+i >= 0 && y+i < image->height)){
                dst_pix[0] = src_pix[0];
                dst_pix[1] = src_pix[1];
                dst_pix[2] = src_pix[2];
            }
        }
    }
}

```

```
}
```

```
void exchange_places(struct exchange_data* space, struct
Png* image, enum error_types* err){
    int lu_x = space->left_up->x;
    int lu_y = space->left_up->y;
    int rd_x = space->right_down->x;
    int rd_y = space->right_down->y;

    if (lu_y>rd_y){
        lu_y = space->right_down->y;
        rd_y = space->left_up->y;
    }

    if (lu_x>rd_x){
        lu_x = space->right_down->x;
        rd_x = space->left_up->x;
    }

    int width_rec = rd_x-lu_x;
    int height_rec = rd_y-lu_y;

    png_bytep* left_up_piece =
(png_bytep*)calloc(height_rec/2, sizeof(png_bytep));
    png_bytep* right_up_piece =
(png_bytep*)calloc(height_rec/2, sizeof(png_bytep));
    png_bytep* left_down_piece =
(png_bytep*)calloc(height_rec/2, sizeof(png_bytep));
    png_bytep* right_down_piece =
(png_bytep*)calloc(height_rec/2, sizeof(png_bytep));

    if (left_up_piece && right_up_piece &&
left_down_piece && right_down_piece){
        get_piece(width_rec/2, height_rec/2,
lu_x, lu_y, left_up_piece, image, err);
        get_piece(width_rec/2, height_rec/2, lu_x
+ width_rec/2, lu_y, right_up_piece, image, err);
        get_piece(width_rec/2, height_rec/2,
lu_x, lu_y + height_rec/2, left_down_piece, image, err);
```

```

        get_piece(width_rec/2, height_rec/2, lu_x
+ width_rec/2, lu_y + height_rec/2, right_down_piece,
image, err);

        if (!strcmp(space->type, "clockwise")){
            paste_piece(lu_x, lu_y,
width_rec/2, height_rec/2, left_down_piece, image);
            paste_piece(lu_x + width_rec/2,
lu_y, width_rec/2, height_rec/2, left_up_piece, image);
            paste_piece(lu_x, lu_y +
height_rec/2, width_rec/2, height_rec/2,
right_down_piece, image);
            paste_piece(lu_x + width_rec/2,
lu_y + height_rec/2, width_rec/2, height_rec/2,
right_up_piece, image);

        } else if (!strcmp(space->type,
"counterclockwise")){
            paste_piece(lu_x, lu_y,
width_rec/2, height_rec/2, right_up_piece, image);
            paste_piece(lu_x + width_rec/2,
lu_y, width_rec/2, height_rec/2, right_down_piece,
image);
            paste_piece(lu_x, lu_y +
height_rec/2, width_rec/2, height_rec/2, left_up_piece,
image);
            paste_piece(lu_x + width_rec/2,
lu_y + height_rec/2, width_rec/2, height_rec/2,
left_down_piece, image);

        } else if (!strcmp(space->type,
"diagonals")){
            paste_piece(lu_x, lu_y,
width_rec/2, height_rec/2, right_down_piece, image);
            paste_piece(lu_x + width_rec/2,
lu_y, width_rec/2, height_rec/2, left_down_piece, image);
            paste_piece(lu_x, lu_y +
height_rec/2, width_rec/2, height_rec/2, right_up_piece,
image);

```

```

        paste_piece(lu_x + width_rec/2,
lu_y + height_rec/2, width_rec/2, height_rec/2,
left_up_piece, image);

    }

    for (int i=0; i<height_rec/2; i++){
        free(left_up_piece[i]);
        free(right_up_piece[i]);
        free(left_down_piece[i]);
        free(right_down_piece[i]);
    }

} else *err = ERR_MEM;

if (left_up_piece)
    free(left_up_piece);
if (right_up_piece)
    free(right_up_piece);
if (left_down_piece)
    free(left_down_piece);
if(right_down_piece)
    free(right_down_piece);
}

void replace_freq_color(struct change_color* my_colors,
struct Png* image, enum error_types* err){
    if (image->color_type == PNG_COLOR_TYPE_RGB){
        struct RGB* freq_color = (struct
RGB*)calloc(1, sizeof(struct RGB));

        int* clr_cnt =
(int*)calloc(NUM_COLORS_RGB, sizeof(int));
        int max_cnt = -1;

        if (clr_cnt && freq_color){
            for (int y=0; y < image->height;
y++){
                png_byte* row = image->row_pointers[y];

```

```

                                for (int x=0; x<image-
>width; x++){
                                png_byte* pixel =
                                &(row[x*3]);

                                int idx =
                                (pixel[0] << 16) | (pixel[1] << 8) | (pixel[2]);
                                clr_cnt[idx]++;

                                if (clr_cnt[idx]
                                > max_cnt){
                                max_cnt =
                                clr_cnt[idx];

                                freq_color->R = pixel[0];
                                freq_color->G = pixel[1];
                                freq_color->B = pixel[2];
                                }
                                }
                                } else {
                                printf("Ошибка памяти\n");
                                *err = ERR_MEM;
                                }

                                for (int y=0; y < image->height; y++){
                                png_byte* line = image-
                                >row_pointers[y];
                                for (int x=0; x < image->width;
                                x++){
                                png_byte* cur_pixel =

                                if (cur_pixel[0] ==
                                freq_color->R && cur_pixel[1] == freq_color->G &&
                                cur_pixel[2] == freq_color->B){
                                cur_pixel[0] =
                                my_colors->new_color->R;

```

```

my_colors->new_color->G;
my_colors->new_color->B;
                                cur_pixel[1] =
                                cur_pixel[2] =
                                }
                                }
                                }
                                free(freq_color);
                                free(clr_cnt);
        } else {
            printf("Формат изображения не RGB\n");
            *err = ERR_IMG_TYPE;
        }
    }

void make_contrast(struct contrast_data* c_data, struct
Png* image){
    for (int y=0; y<image->height; y++){
        png_byte* row = image->row_pointers[y];
        for (int x=0; x<image->width; x++){
            png_byte* pix = &(row[x*3]);

            int red = (c_data->alpha * pix[0]
+ c_data->beta)/1;
            int green = (c_data->alpha *
pix[1] + c_data->beta)/1;
            int blue = (c_data->alpha *
pix[2] + c_data->beta)/1;

            if (red <= 255)
                pix[0] = red;
            else pix[0] = 255;

            if (green <= 255)
                pix[1] = green;
            else pix[1] = 255;

            if (blue <= 255)
                pix[2] = blue;
            else pix[2] = 255;

```

```

    }
}

int main(int argc, char** argv){

    struct options_flags flags = {0};

    struct option options[] = {
        {"input", required_argument, NULL, 'i'},
        {"output", required_argument, NULL, 'o'},
        {"info", no_argument, NULL, OPT_INFO},
        {"help", no_argument, NULL, 'h'},
        {"contrast", no_argument, NULL, 'R'},

        {"square", no_argument, NULL, 'S'},
        {"left_up", required_argument, NULL,
'l'},
        {"side_size", required_argument, NULL,
's'},
        {"thickness", required_argument, NULL,
't'},
        {"color", required_argument, NULL, 'C'},
        {"fill", no_argument, NULL, 'f'},
        {"fill_color", required_argument, NULL,
'c'},
        {"exchange", no_argument, NULL, 'e'},
        {"right_down", required_argument, NULL,
'r'},
        {"exchange_type", required_argument,
NULL, 'T'},
        {"freq_color", no_argument, NULL, 'F'},
        {"alpha", required_argument, NULL, 'a'},
        {"beta", required_argument, NULL, 'b'},
        {0, 0, 0, 0}
    };

    char optstring[SZ_OPTSTR] =
    "i:o:hRSl:s:t:C:fc:e:r:T:Fa:b:";
    int optidx = 0;
    int opt;

```



```

enum error_types err = 0;

int checker = 0;
int help_opt = 0;
int info_opt = 0;

char* img_iname = (char*)calloc(SZ_INPUT,
sizeof(char));
char* img_otype = (char*)calloc(SZ_OUTPUT,
sizeof(char));

if (img_otype)
    strcpy(img_otype, "out.png");
else err = ERR_MEM;

struct point* l_point = (struct point*)calloc(1,
sizeof(struct point));
struct point* r_point = (struct point*)calloc(1,
sizeof(struct point));
struct RGB* color = (struct RGB*)calloc(1,
sizeof(struct RGB));
struct RGB* fill_color = (struct RGB*)calloc(1,
sizeof(struct RGB));

struct square_data* my_square = (struct
square_data*)calloc(1, sizeof(struct square_data));
struct exchange_data* exchange_data = (struct
exchange_data*)calloc(1, sizeof(struct exchange_data));
struct change_color* my_freq_color = (struct
change_color*)calloc(1, sizeof(struct change_color));
struct contrast_data* my_contrast = (struct
contrast_data*)calloc(1, sizeof(struct contrast_data));

struct Png* my_image = (struct Png*)calloc(1,
sizeof(struct Png));

while ((opt = getopt_long(argc, argv, optstring,
options, &optidx)) != -1){
    switch(opt){
        case 'i':

```

```

                                if (optind < argc &&
argv[optind][0] != '-') {
                                printf("Опция --
input принимает только один аргумент.\n%s и следующие
аргументы будут проигнорированы.\n", argv[optind]);
                                err = ERR_OPT;
                                } else {
                                img_iname =
realloc(img_iname, strlen(optarg));

                                if (img_iname){

strcpy(img_iname, optarg);

                                if
(read_image(my_image, img_iname, &err))

                                else {

printf("Ошибка чтения изображения\n");

err = ERR_IMG_READ;

                                }

                                } else err =

ERR_MEM;

                                }
                                break;
                                case 'o':
                                if ((optind < argc &&
argv[optind][0] != '-') && optind != argc-1){
                                printf("Опция --
output принимает только один аргумент.\n%s и следующие
аргументы будут проигнорированы.\n", argv[optind]);
                                err = ERR_OPT;
                                } else {
                                img_otype =
realloc(img_otype, strlen(optarg)+1);

                                if (img_otype){

```

```

                                                                    if
(optarg[0] != '-')
strcpy(img_ename, optarg);
                                                                    else {
printf("Не введено имя выходного файла\n");
err = ERR_OPT;
                                                                    } else {
printf("Ошибка памяти\n");
                                                                    err =
ERR_MEM;
                                                                    }
                                                                    }
                                                                    break;
case 'h':
    if ((optind < argc &&
argv[optind][0] == '-') || optind == argc || optind ==
argv-1){
                                                                    help_opt = 1;
                                                                    printf("Course
work for option 4.16, created by Alina Kiseleva\n");
                                                                    print_help();
    } else {
                                                                    printf("Опция --
help не принимает аргументов.\n%s и последующие аргументы
будут проигнорированы.\n", argv[optind]);
                                                                    err = ERR_OPT;
    }
                                                                    break;
case 'R':
    if (my_contrast)
                                                                    flags.contr_flag
= 1;
                                                                    if ((optind < argc &&
argv[optind][0] != '-') && optind != argv-1){

```

```

                                printf("Опция --
contrast не принимает аргументов.\n%s и последующие
аргументы будут проигнорированы.\n", argv[optind]);
                                err = ERR_OPT;
                                }
                                break;
                                case 'S':
                                    if (my_square)
                                        flags.sqr_flag =
1;
                                    if ((optind < argc &&
argv[optind][0] != '-') && optind != argc-1){
                                        printf("Опция --
square не принимает аргументов.\n%s и последующие
аргументы будут проигнорированы.\n", argv[optind]);
                                        err = ERR_OPT;
                                    }
                                    break;
                                case 'e':
                                    if (exchange_data)
                                        flags.exchg_flag
= 1;
                                    if ((optind < argc &&
argv[optind][0] != '-') && optind != argc-1){
                                        printf("Опция --
exchange не принимает аргументов.\n%s и последующие
аргументы будут проигнорированы.\n", argv[optind]);
                                        err = ERR_OPT;
                                    }
                                    break;
                                case 'F':
                                    if (my_freq_color)
                                        flags.fq_clr_flag
= 1;
                                    if ((optind < argc &&
argv[optind][0] != '-') && optind != argc-1){
                                        printf("Опция --
freq_color не принимает аргументов.\n%s и последующие
аргументы будут проигнорированы.\n", argv[optind]);
                                        err = ERR_OPT;
                                    }
                                }

```

```

                                break;
                                case 'a':
                                    if ((optind < argc &&
argv[optind][0] != '-') && optind != argc-1){
                                        printf("Опция --
alpha принимает только один аргумент.\n%s и следующие
аргументы будут проигнорированы.\n", argv[optind]);
                                        err = ERR_OPT;
                                    } else {
                                        if (my_contrast){
                                            checker =
sscanf(optarg, "%lf", &my_contrast->alpha);
                                            if
(ch checker == 1 && my_contrast->alpha > 0)
flags.a_flag = 1;
                                            else {
printf("Коэффициент изменения контрастности alpha задан
неверно\n");
err = ERR_COORDS;
                                }
                                } else err =
ERR_MEM;
                                }
                                break;
                                case 'b':
                                    if ((optind < argc &&
argv[optind][0] != '-') && optind != argc-1){
                                        printf("Опция --
beta принимает только один аргумент.\n%s и следующие
аргументы будут проигнорированы.\n", argv[optind]);
                                        err = ERR_OPT;
                                    } else {
                                        if (my_contrast){
                                            checker =
sscanf(optarg, "%d", &my_contrast->beta);
                                            if
(ch checker == 1)

```

```

flags.b_flag = 1;
                                                                    else {
printf("Сдвиг контрастности beta задан неверно\n");
err = ERR_COORDS;
                                                                    }
ERR_MEM;
                                                                    } else err =
                                                                    }
                                                                    break;
case 'l':
    if ((optind < argc &&
argv[optind][0] != '-') && optind != argc-1){
        printf("Опция --
left_up принимает только один аргумент.\n%s и следующие
аргументы будут проигнорированы.\n", argv[optind]);
        err = ERR_OPT;
    } else {
        if (l_point &&
my_square && exchange_data){
                                                                    checker =
sscanf(optarg, "%d.%d", &l_point->x, &l_point->y);
                                                                    if
(checker == 2){
my_square->left_up = l_point;
exchange_data->left_up = l_point;
flags.lft_up_flag = 1;
                                                                    } else {
printf("Координаты левой верхней точки заданы неверно\
n");
err = ERR_COORDS;
                                                                    }
                                                                    } else {

```

```

printf("Ошибка памяти\n");
err =
ERR_MEM;
}
}
break;
case 's':
if ((optind < argc &&
argv[optind][0] != '-') && optind != argc-1){
printf("Опция --
side_size принимает только один аргумент.\n%s и следующие
аргументы будут проигнорированы.\n", argv[optind]);
err = ERR_OPT;
} else {
if (my_square){
checker =
sscanf(optarg, "%d", &my_square->side_size);
if
(checker == 1 && my_square->side_size > 0){
flags.sd_sz_flag = 1;
} else {
printf("Сторона задана неверно\n");
err = ERR_OPT;
} else {
}
}
printf("Ошибка памяти\n");
err =
ERR_MEM;
}
}
break;
case 't':
if ((optind < argc &&
argv[optind][0] != '-') && optind != argc-1){

```

```

                                printf("Опция --
thickness принимает только один аргумент.\n%s и следующие
аргументы будут проигнорированы.\n", argv[optind]);
                                err = ERR_OPT;
                                } else {
                                    if (my_square){
                                        checker =
sscanf(optarg, "%d", &my_square->thickness);
                                        if
(cchecker == 1 && my_square->thickness >= 0){
flags.thkns_flag = 1;
                                        } else {

printf("Толщина задана неверно\n");
err = ERR_OPT;

                                } else {
                                    }
                                }
                                }
                                break;
                                case 'C':
                                    if ((optind < argc &&
argv[optind][0] != '-') && optind != argc-1){
                                        printf("Опция --
color принимает только один аргумент.\n%s и следующие
аргументы будут проигнорированы.\n", argv[optind]);
                                        err = ERR_OPT;
                                    } else {
                                        if (color &&
my_square && my_freq_color){
                                            checker =
sscanf(optarg, "%d.%d.%d", &color->R, &color->G, &color-
>B);

```



```

                                if
((checker == 3) && check_color(color->R, color->G, color-
>B)){
flags.clr_flag = 1;
my_square->color = color;
my_freq_color->new_color = color;
                                } else {

printf("Цвет задан неверно\n");
err = ERR_COLOR;

                                } else {
}

printf("Ошибка памяти\n");
                                err =
ERR_MEM;

                                }
                                }
                                break;
case 'f':
    if (my_square){
        flags.fill_flag =
1;
        my_square->fill =
(bool)1;
    } else {
        printf("Ошибка
памяти\n");
        err = ERR_MEM;
    }
    break;
case 'c':
    if ((optind < argc &&
argv[optind][0] != '-') && optind != argc-1){
        printf("Опция --
fill_color принимает только один аргумент.\n%s и

```

```

следующие аргументы будут проигнорированы.\n",
argv[optind]);

                                err = ERR_OPT;
                                } else {
                                    if (fill_color){
                                        int
checker = sscanf(optarg, "%d.%d.%d", &fill_color->R,
&fill_color->G, &fill_color->B);
                                        if
((checker == 3) && check_color(fill_color->R, fill_color-
>G, fill_color->B)){
if (my_square){
my_square->fill_color = fill_color;
flags.fill_clr_flag = 1;
}
else if (!my_square){
printf("Ошибка памяти\n");
err = ERR_MEM;
} else {
printf("Цвет заливки задан неверно\n");
err = ERR_COLOR;
} else {
printf("Ошибка памяти\n");
err =
ERR_MEM;
}
}
break;
case 'r':
if ((optind < argc &&
argv[optind][0] != '-') && optind != argc-1){

```

```

                                printf("Опция --
right_down принимает только один аргумент.\n%s и
следующие аргументы будут проигнорированы.\n",
argv[optind]);
                                err = ERR_OPT;
                                } else {
                                    if (r_point){
                                        int
checker = sscanf(optarg, "%d.%d", &r_point->x, &r_point->y);
                                if
                                (checker == 2){
                                    if (exchange_data){
                                        exchange_data->right_down = r_point;
                                        flags.rt_dwn_flag = 1;
                                }
                                else {
                                    printf("Ошибка памяти\n");
                                    err = ERR_MEM;
                                } else {
                                printf("Координаты правой нижней точки заданы неверно\n
n");
                                err = ERR_COORDS;
                                } else {
                                printf("Ошибка памяти\n");
                                err =
                                ERR_MEM;
                                }
                                }
                                break;
                                case 'T':

```

```

                                if ((optind < argc &&
argv[optind][0] != '-') && optind != argc-1){
                                printf("Опция --
exchange_type принимает только один аргумент.\n%s и
следующие аргументы будут проигнорированы.\n",
argv[optind]);
                                err = ERR_OPT;
                                } else {
                                    if
(exchange_data){
                                if (!
strcmp(optarg, "clockwise") || !strcmp(optarg,
"counterclockwise") || !strcmp(optarg, "diagonals")){
exchange_data->type = optarg;
flags.exchg_t_flag = 1;
                                } else {
printf("Введенного способа обмена частей не существует\
n");
err = ERR_EXCH_TYPE;
                                }
                                } else {
printf("Ошибка памяти\n");
err =
ERR_MEM;
                                }
                                }
                                break;
                                case OPT_INFO:
                                if ((optind < argc &&
argv[optind][0] == '-') || optind == argc-1){
                                info_opt = 1;
                                } else {
                                printf("Опция --
info не принимает аргументов.\n%s и последующие аргументы
будут проигнорированы.\n", argv[optind]);
                                err = ERR_OPT;

```

```

        }
        break;
    case '?':
    default:
        printf("Введенного флага
не существует.\n");

        err = ERR_OPT;
        break;
    }
}

/* Проверка на то, что последним подано имя
входного файла */
if (!flags.iname_flag || !img_iname){

    if (argv[argc-1][0] != '-'){
        img_iname = realloc(img_iname,
strlen(argv[argc-1]));

        if (img_iname){
            strcpy(img_iname,
argv[argc-1]);

            if (read_image(my_image,
img_iname, &err))
                flags.iname_flag
= 1;

            } else err = ERR_MEM;

        } else if (!help_opt)
            err = ERR_INPUT_NAME;

    }

    if (img_iname != NULL && strcmp(img_iname,
img_oiname) == 0){
        printf("Имена входного и выходного файлов
совпадают\n");
        err = ERR_OUTPUT_NAME;
    }

    if (!flags.err && !help_opt)

```

```

        printf("Course work for option 4.16,
created by Alina Kiseleva\n");

    if (info_opt)
        print_png_info(my_image);

    if (err) flags.err = 1;

    int is_valid = check_input_data(flags);

    if (is_valid){
        if (is_valid == 1)
            draw_square(my_square, my_image);
        else if (is_valid == 2)
            exchange_places(exchange_data,
my_image, &err);
        else if (is_valid == 3)
            replace_freq_color(my_freq_color,
my_image, &err);
        else if (is_valid == 4)
            make_contrast(my_contrast,
my_image);
        write_image(my_image, img_otype, &err);
    }

    if (img_iname)
        free(img_iname);
    if (img_otype)
        free(img_otype);
    if (l_point)
        free(l_point);
    if (r_point)
        free(r_point);
    if (color)
        free(color);
    if (fill_color)
        free(fill_color);
    if (my_square)
        free(my_square);
    if (exchange_data)
        free(exchange_data);

```

```
    if (my_freq_color)
        free(my_freq_color);
    if (my_image)
        free(my_image);
    if (my_contrast)
        free(my_contrast);

    return err;
}
```